Multilingual Support in 4D Databases with XLIFF Integration

By Karim MEGHRAOUI, Technical Support Engineer, 4D Morocco.

Technical Note 24-11

Table of Contents

Table of Contents 2
Abstract 3
Introduction 3
Requirements 3
4D 20.4 LTS or later version (20 R5, 20 R6 bêta….) 3
Language certification matrix 3
Unicode 3
Compatibility Notes on Resource Management 4
OS and Application Language Settings 4
1. 4D Programming Language (Using System Regional Settings) 4
2. Customizing Language Settings for 4D Applications on macOS 5
XLIFF Architecture 6
1. Overview of the XLIFF Format 7
2. Structure of an XLIFF File 7
Example of Implementing an XLIFF File 9
1. Implementing XLIFF in a 4D Application 11
2. How XLIFF Works 11
Deploying 4D Bases with XLIFF 13
Scenario 1: 14
Scenario 2: 14
Scenario 3: 14
How to Manipulate XLIFF Files with the Get Localized String Command 15
Set Database Localization command for Current Language Control 16
Asian and Arabic Languages in 4D Databases 18
1. Asian Languages 18
2. Arabic Language 18
3. Importing Arabic Data into a 4D Database 20
Conclusion 20

Abstract

Designing a multilingual application is often crucial to reaching a diverse audience. Localization refers to the ability to adapt software and applications to different languages, cultures, and regional formats. This adaptation can take place at various levels, ranging from the simple translation of displayed texts to more complex adjustments, such as number and date formats, keyboard layout, legal requirements specific to each region, and using particular symbols and icons.

Introduction

This technical note focuses on translating the user interface into multiple languages. It examines how to adapt a 4D database for various languages, specifically using the 4D command, SET DATABASE LOCALIZATION. It also details how to adjust the interface to make it accessible in different languages and enable the application to dynamically switch between these languages when opened. The goal is to ensure a consistent and seamless user experience regardless of the user's language or culture.

Requirements

The following elements are required to use XLIFF in a 4D application:

4D 20.4 LTS or later version (20 R5, 20 R6 bêta….)



Language certification matrix

The developer must check the most recent language certification matrix to confirm that the language is supported. Compatibility matrices are accessible on the official 4D site and are organized by supported versions in the related resources section.

Unicode

In a 4D application, Unicode is used to manage and display text in different languages, enabling the creation of truly international applications. Using Unicode, 4D can store, manipulate, and search text using a wide range of characters from different alphabets, ensuring full multilingual support and avoiding problems with data corruption. It also

enables seamless interoperability with other systems and applications, while offering the possibility of integrating special symbols, enriching the user experience.

Compatibility Notes on Resource Management

4D has updated its resource management to match Apple's guidelines and the latest macOS versions. Newer versions of 4D use standard files like XLIFF for translations and .png for images, replacing old resource files. 4D now provides new tools for handling translations with XLIFF files while keeping compatibility with existing systems. "Resources" now refers to essential files for translating application interfaces. These should be placed in a "Resources" folder next to the database structure file (.4db or .4dc) or next to the Project folder for Project mode databases. This folder must include all files needed for translation and customization, such as images, text files, and XLIFF files.

Note: Newer 4D versions no longer use .RSR (structure resources) and .4DR (data resources) files by default. It's advised to update converted databases and remove any references to 4D's old resources.

OS and Application Language Settings

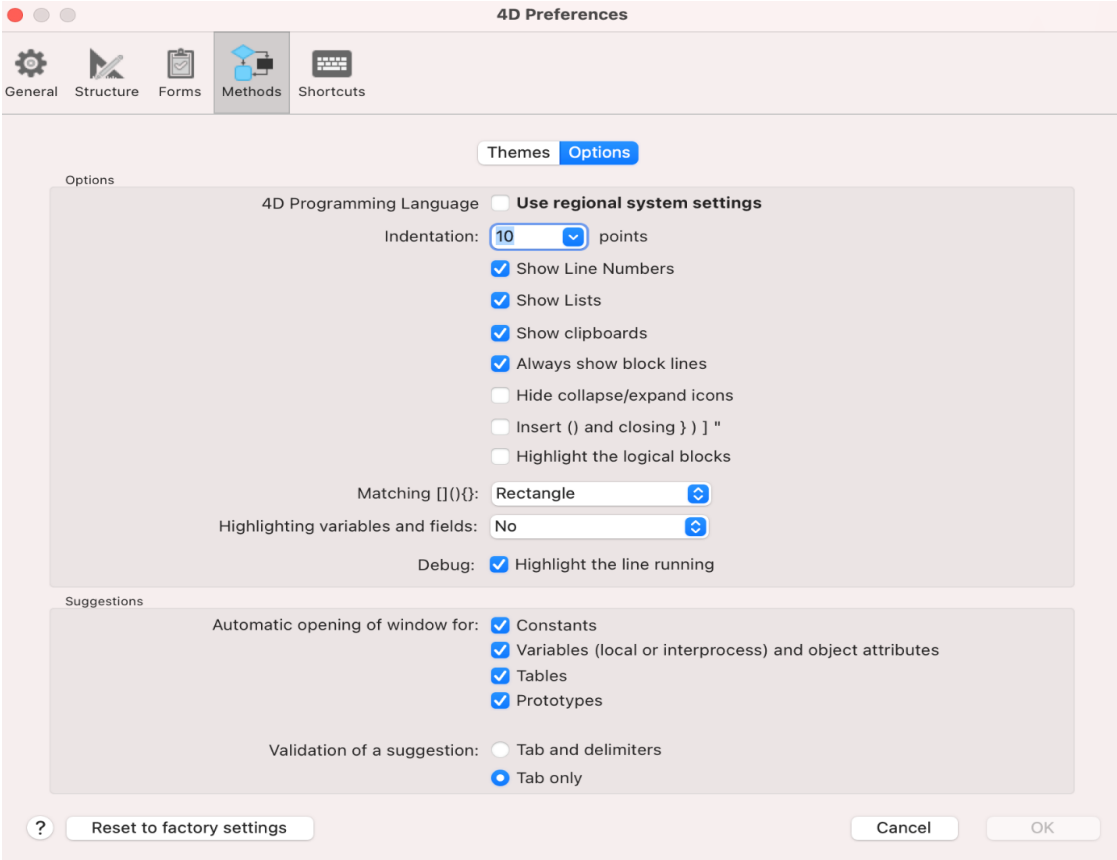
1. 4D Programming Language (Using System Regional Settings)

Starting from version 15 of 4D, the 4D method editor defaults to the international mode "English-US," regardless of the application's language settings or system regional settings. This approach ensures that code is unaffected by regional variations that might disrupt interpretation, such as date formats, across different versions of the 4D application. Additionally, in the French version of 4D, commands and constants are also in "English-US."

This default setting offers several benefits for 4D developers:

Code Sharing: It simplifies code sharing among developers without concerns about location, regional settings, or the version of 4D used. 4D methods can be easily shared via copy/paste or saved in text files without compatibility issues.
Version Control Compatibility: It ensures that 4D methods are compatible with version control tools, which often require exports to be independent of regional or language settings.

This default setting can be disabled via the "Use regional system settings" option in the 4D Preferences dialog:



This setting allows enabling or disabling the "international" code parameters for the local 4D application.

Unchecked (default): "English-US" settings and English programming language are used in 4D methods.
Checked: Regional settings are used in 4D methods.

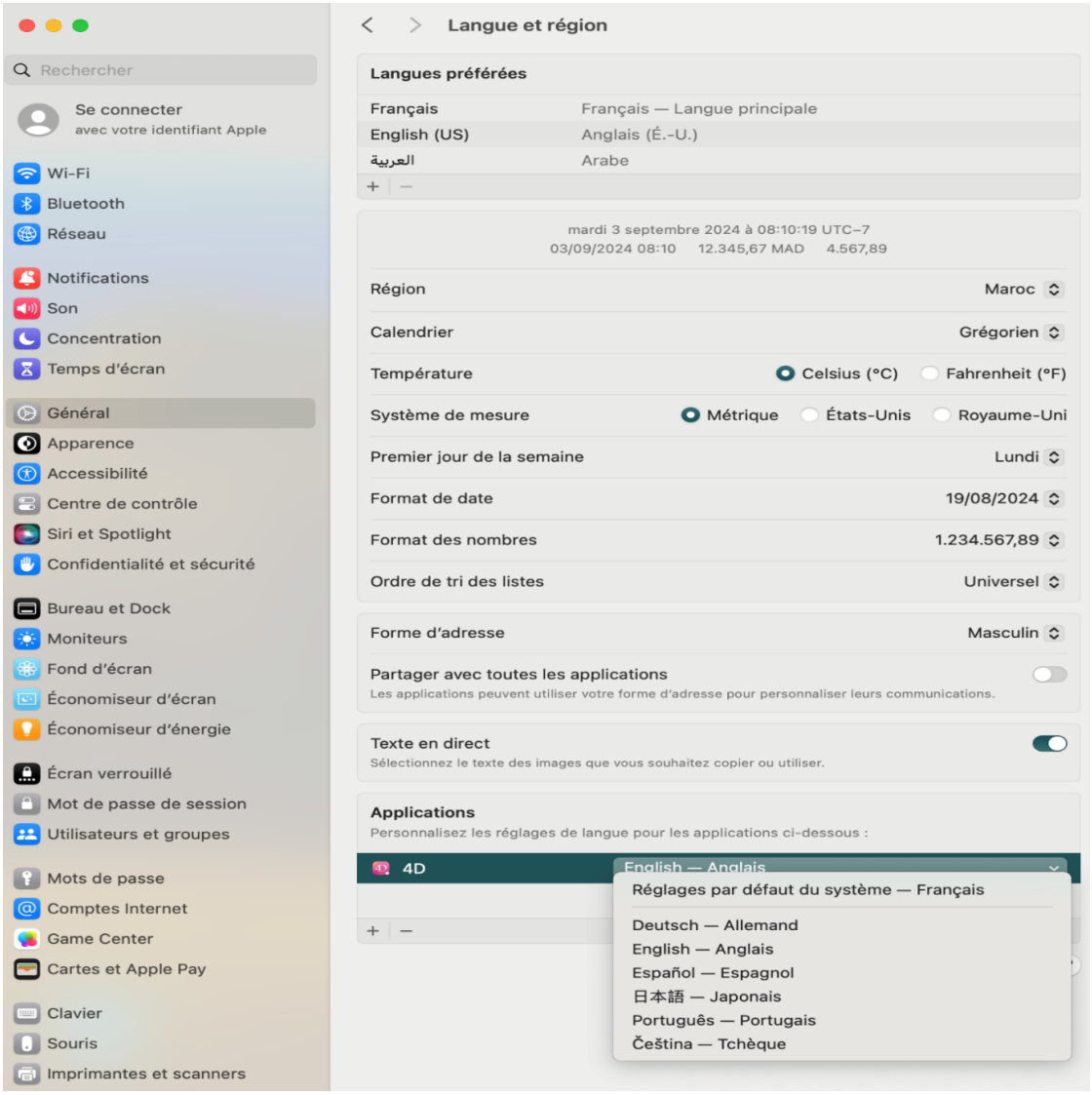
If this option is changed, the 4D application must be restarted for the change to take effect.

2. Customizing Language Settings for 4D Applications on macOS

macOS allows users to set a language for the operating system and a specific language for each installed application. To activate this option, follow these steps:

1. On Mac, choose the Apple menu > System Settings, click on General in the sidebar, then select Language & Region on the right. (It might be necessary to scroll down.)
2. Go to Applications, then perform the following actions:

To choose the language for a specific 4D application: Click the Add button, select the version of the 4D application used, choose a language from the local menus, and then click Add.



To change the language of a 4D application from the list: Select the 4D application, then choose a different language from the local menu.
To remove a 4D application from the list: Select the 4D application, then click the Remove button. The application will revert to the default language.

Note: If the 4D application is open, it must be restarted for the changes to take effect.

XLIFF Architecture

To ensure an application adapts easily to different languages and cultures, it is essential to translate all user-visible elements. In a 4D application, this process is streamlined using XLIFF (.xlf) files, which contain all the necessary translations. The XLIFF format, an XML standard, is designed to simplify the management of multilingual content in software.

XLIFF files centralize translations, facilitating the management of linguistic adaptation. The translation process involves converting texts and creating specific keys for each element to be translated. These keys link each text in the application to its corresponding translation in the XLIFF files, ensuring consistency and accuracy while simplifying updates.

Using XLIFF files for the linguistic adaptation of a 4D application offers several advantages:

Efficient management of multilingual content
Reduced risk of translation errors
Consistent user experience

This approach allows developers to focus on other aspects of development while providing an application that is accessible and suitable for a global audience.

1. Overview of the XLIFF Format

XLIFF is an XML format standardized by OASIS (Organization for the Advancement of Structured Information Standards) for the exchange of translation data. It structures source texts and translations into distinct translation units, each consisting of the source text and its corresponding translation. This format facilitates the language adaptation process by enabling centralized and structured management of translations.

XLIFF files are essential for efficient translation management, especially in complex applications that support multiple languages. The XLIFF format can be utilized for:

- Object names in the form editor's property list
- Window titles in the form properties dialog
- Static texts, such as labels for checkboxes, buttons, etc.
- Menus
- Help Tips

2. Structure of an XLIFF File

An XLIFF file consists of several key elements:

- <xliff>: The root element that defines the version of XLIFF used.
- <file>: A container for a source document, specifying the source and target languages, as well as the data type.
- <body>: Contains translation units.
- <trans-unit>: An individual translation unit that includes the source text and its translation.

Below is an example of an XLIFF file:

```
<?xml version="1.0" encoding="UTF-8"?>

<!--
Translation file for 4D products.
Copyright(C)2006 4D SA.
This file is distributed under the same license as the 4D product package.
-->

<!DOCTYPE xliff PUBLIC "-//XLIFF//DTD XLIFF//EN" "http://www.oasis-open.org/committees/xliff/documents/xliff.dtd">

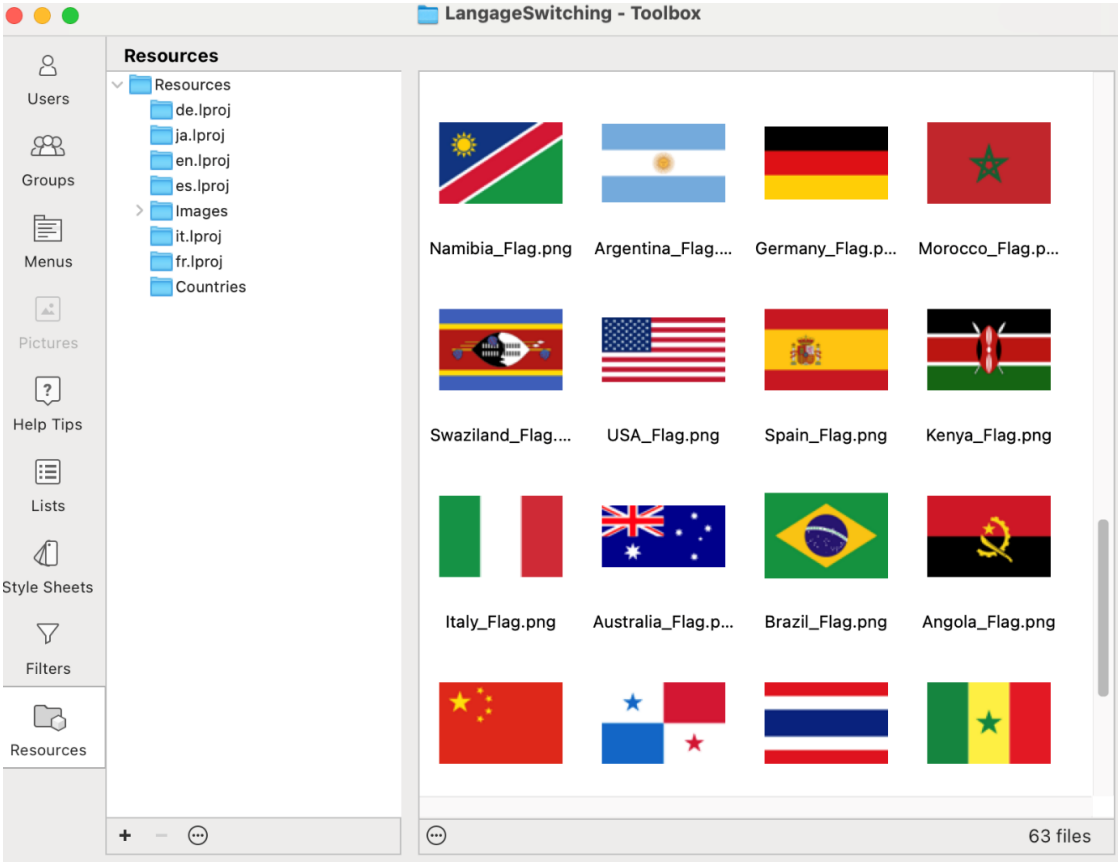
<xliff version="1.0">
  <file build-num="9.0" datatype="x-STR#" date="2006-04-20T12:13:41Z" original="4th Dimension" product-name="4th Dimension" product-version="v20" source-language="en-US" target-language="ja">
    <body>
      <group id="1" resname="SQLAliases">
        <trans-unit id="2" resname="How Do I...">
          <source>How Do I...</source>
          <target>教えて...</target>
        </trans-unit>
        <trans-unit id="3" resname="...switch language dynamically?">
          <source>...switch language dynamically?</source>
          <target>言語を切り替える...?</target>
        </trans-unit>
        <trans-unit id="4" resname="Demo">
          <source>Demo</source>
          <target>デモ</target>
        </trans-unit>
        <trans-unit id="5" resname="Open single dialog">
          <source>Open single dialog</source>
          <target>ひとつの言語を表示する</target>
        </trans-unit>
        <trans-unit id="6" resname="Open all in processes">
          <source>Open all in processes</source>
          <target>すべての言語を表示する</target>
        </trans-unit>
        <trans-unit id="7" resname="Reveal lang files">
          <source>Reveal lang files</source>
          <target>言語ファイルを表示する</target>
        </trans-unit>
        <trans-unit id="8" resname="Selected langage is applied on newly opened dialogs">
          <source>Selected langage is applied on newly opened dialogs</source>
          <target>選択した言語は新しく開かれたダイアログに反映されます。</target>
        </trans-unit>
        <trans-unit id="9" resname="Change Language">
          <source>Change Language</source>
          <target>言語の切り替え</target>
        </trans-unit>
        <trans-unit id="10" resname="SET DATABASE LOCALIZATION(Language(;*))">
          <source>SET DATABASE LOCALIZATION(Language(;*))</source>
          <target>SET DATABASE LOCALIZATION(言語(;*))</target>
        </trans-unit>
        <trans-unit id="11" resname="text::set database localization detail">
          <source>SET DATABASE LOCALIZATION(Language)
```

Individual text elements are stored in <trans-unit> elements within the XLIFF file. Each <trans-unit> element has an identifier and a resname, which can then be used in 4D to display the corresponding text.

Texts contained in XLIFF files can be automatically utilized if they are placed in lproj folders within the database resource directory.

When creating XLIFF files for different languages, it is crucial to maintain consistency in id and resname values across languages. For example, if a welcome message is to be available in English, Spanish, and French, the same id must be used in the form. This id must be present in all three XLIFF files created for these languages.

Below is the Resources folder illustrating the location of the various XLIFF files within the database:



Example of Implementing an XLIFF File

Consider a scenario where a 4D application needs to support both English and Spanish. Below is an example demonstrating how to implement an XLIFF file to translate the days of the week from English to Spanish using unique resname attributes such as day_1, day_2, and so on, to identify each day.

In this example, the XLIFF file includes seven <trans-unit> elements, each representing a day of the week. The resname attribute is used to uniquely identify each day in the 4D application.

Here's a breakdown of the relevant elements:

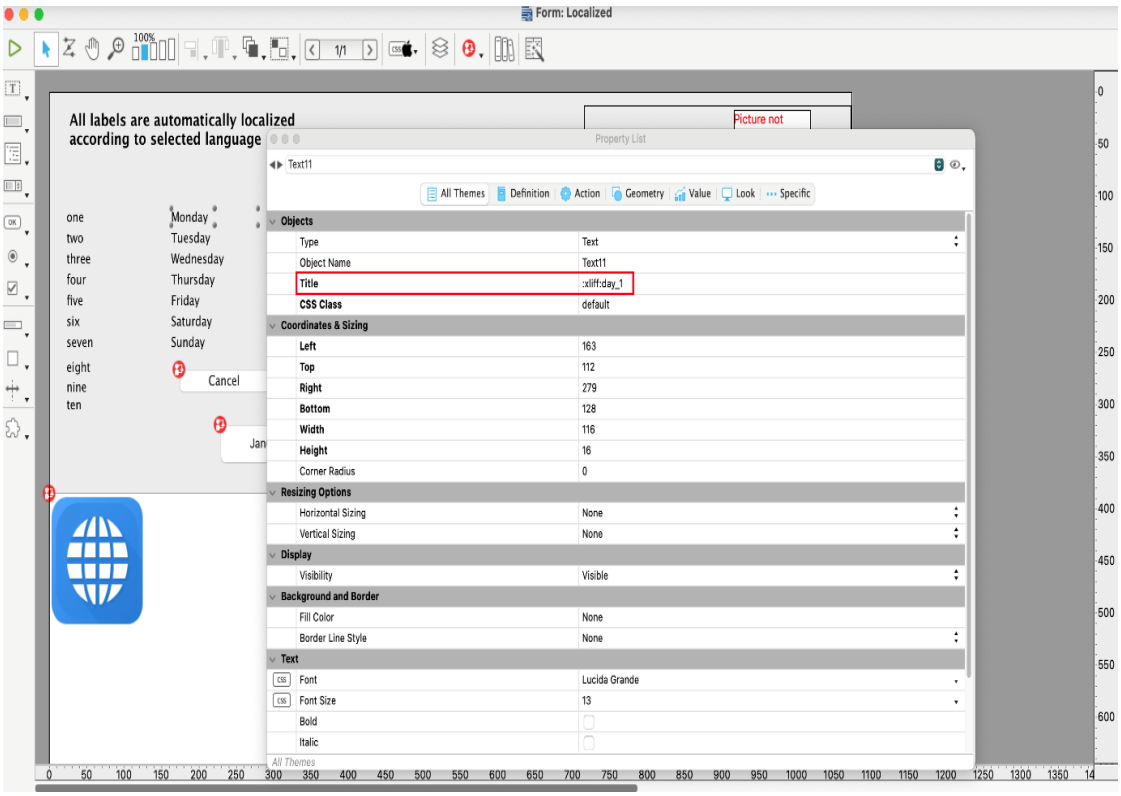
- source-language: Specifies the source language of the text to be translated (e.g., English, "en-US").
- target-language: Specifies the target language for the translation (e.g., Spanish, "es").
- resname: A unique identifier for each translated element (e.g., "day_1" for Monday).
- source: The original text in the source language (e.g., "Monday").
- target: The corresponding translation in the target language (e.g., "Lunes").

Each day of the week is associated with a unique reference via the resname attribute (day_1 for Monday, day_2 for Tuesday, etc.).

Below is an example snippet of an XLIFF file illustrating the translation of days of the week:

```
<group resname="DayNames">
   <trans-unit id="1" resname="day_1">
      <source>Monday</source>
      <target>Lunes</target>
   </trans-unit>
   <trans-unit id="2" resname="day_2">
      <source>Tuesday</source>
      <target>Marte</target>
   </trans-unit>
   <trans-unit id="3" resname="day_3">
      <source>Wednesday</source>
      <target>Miércoles</target>
   </trans-unit>
   <trans-unit id="4" resname="day_4">
      <source>Thursday</source>
      <target>Jueves</target>
   </trans-unit>
   <trans-unit id="5" resname="day_5">
      <source>Friday</source>
      <target>Viernes</target>
   </trans-unit>
   <trans-unit id="6" resname="day_6">
      <source>Saturday</source>
      <target>Sábado</target>
   </trans-unit>
   <trans-unit id="7" resname="day_7">
      <source>Sunday</source>
      <target>Domingo</target>
   </trans-unit>
</group>
```

The 4D developer can then incorporate these references into the application as follows:



Instead of directly specifying the name of the day in the title (e.g., “Monday”), the developer will use an XLIFF reference such as :xliff:day_1 in the “title” object property. When the application is run, 4D will retrieve the appropriate translation from the XLIFF file based on the active language of the application user.

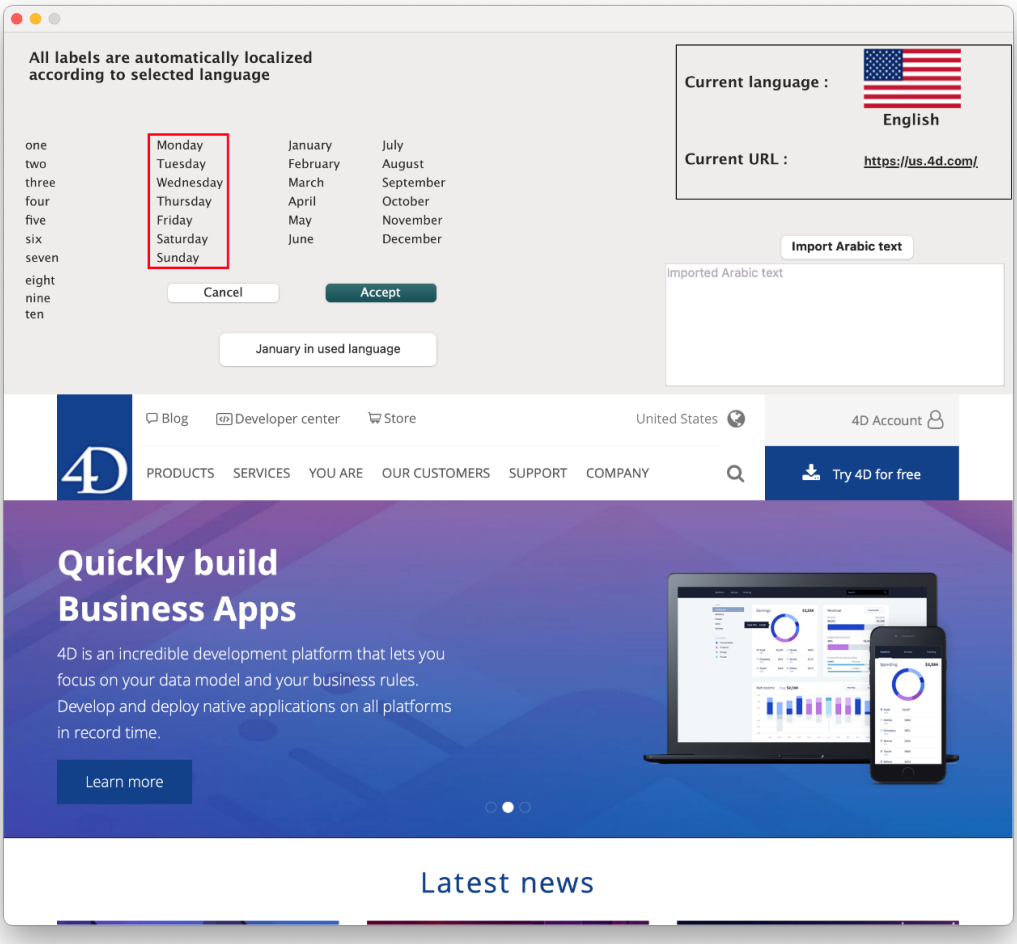
1. Implementing XLIFF in a 4D Application

At runtime, 4D will use these identifiers to display the names of the days of the week according to the user's active language. For example, if the language is set to Spanish, the text “Monday” will automatically be replaced by “Lunes” in the user interface.

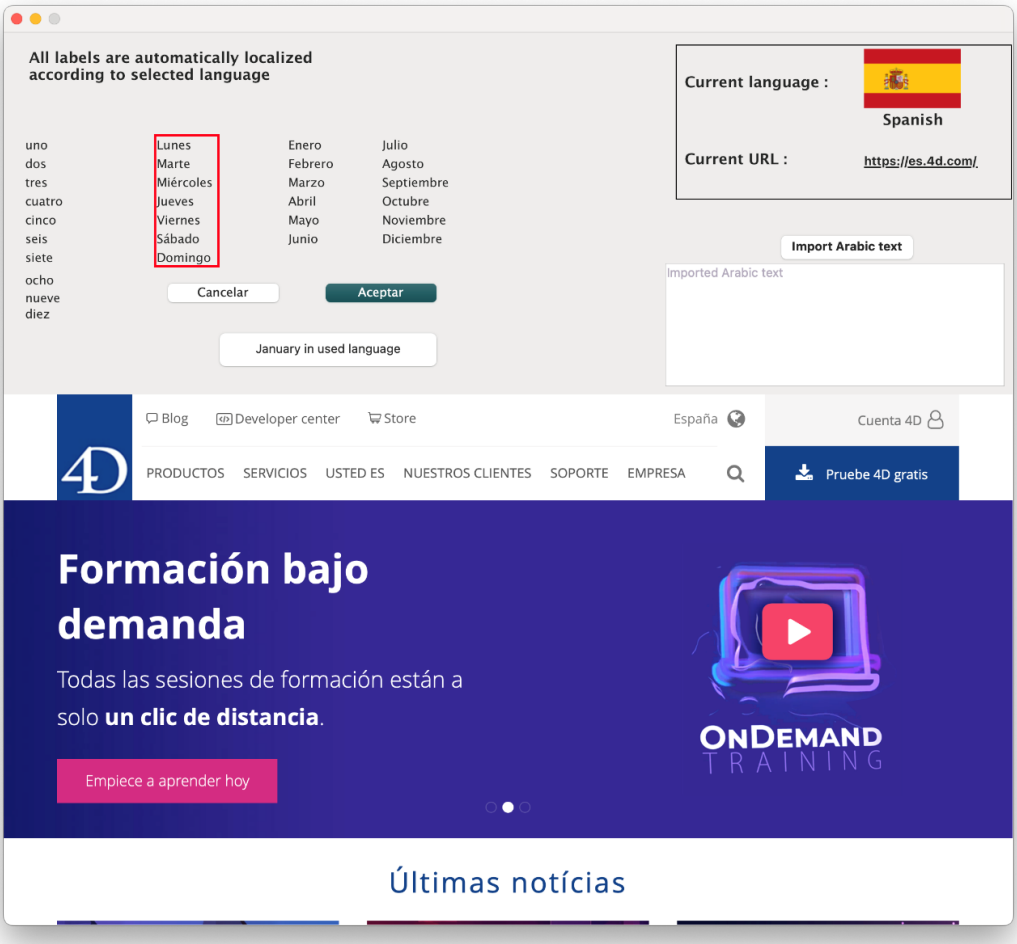
This structure allows developers to easily add new languages or modify existing translations by simply editing the XLIFF file, without requiring any changes to the application's source code.

2. How XLIFF Works

In the application: If the active language is English, :xliff:day_1 will display “Monday,” :xliff:day_2 will display “Tuesday,” and so on.



If the application's active language is Spanish, these same references will display “Lunes”, “Martes”, etc.and so on, respectively:



Deploying 4D Bases with XLIFF

By configuring an application to use linguistic resources (XLIFF) instead of static text in its interface, 4D can dynamically display the application in multiple languages. The default setting allows 4D to select the appropriate language during database startup or when a client connects to the server.

The `Get database localization` command is used to identify the current language defined for the database.

The choice of language is defined by examining the lproj folders and following the rule of this order of priority:

The language selection is determined by examining the lproj folders and following this priority order:

1. **System language:** On macOS, if multiple languages are set in order of preference, 4D will use this language configuration.
2. **Current language of the 4D application:** The language set within the application itself.
3. **English:** If neither of the above options is available, English will be used as the default language.

4. **First language found in the Resources directory:** The application will default to the first language it finds in the Resources directory if none of the previous options apply.

4.1 Language deployment examples:

Scenario 1:

An application contains the following folders in the Resources folder:

- en.lproj
- es.lproj
- fr.lproj

The application operates in an engined Client/Server environment and is integrated with an English version of 4D (English was selected as the default language during 4D installation).

- **Remote Client A** connects from a machine where the system language (OS) is French.

  Result: The application runs in French for Client A.

  Explanation: The fr.lproj folder matches the system language (rule 1).

- **Remote Client B** connects from a computer where the system language is German.

  Result: The application runs in English for Customer B.

  Explanation: The de.lproj folder does not exist, so rule 1 does not apply. The default language of the application is English (rule 3).

Scenario 2:

An application contains the following folders in the Resources directory:

- en.lproj
- es.lproj

The application is an engined Client/Server with a Japanese version of 4D.

- **The remote client** connects from a machine where the system language is Italian.

  Result: The application runs in English for the client.

  Explanation: Since there is no folder for Italian and the application's current language settings do not match Italian or Japanese (rule 1 or rule 2), the default language (English) is used (rule 3).

Scenario 3:

An application contains the following folders in the Resources directory:

- es.lproj
- fr.lproj

The application is an engined Client/Server with a German version of 4D.

- **Remote client A** connects from a machine where the system language is Portuguese.

  Result: The application runs in Spanish for remote client A.

  Explanation: Since there is no match for rules 1 (system language), 2 (current application language), or 3 (default language), the application defaults to the first available language listed in the Resources directory, which is Spanish (rule 4).

Note: Regardless of the language in which the 4D database is localized, the 4D user interface elements—such as the query editor, sort dialogs, and Quick Reports editor—will always appear in the language in which the application was originally built. To ensure these elements integrate correctly with the application's target language, a custom interface may need to be developed.

How to Manipulate XLIFF Files with the Get Localized String Command

To retrieve text programmatically from an XLIFF file, use the **Get Localized String** command. This command retrieves the translation of a text string specified by resName for the current language.

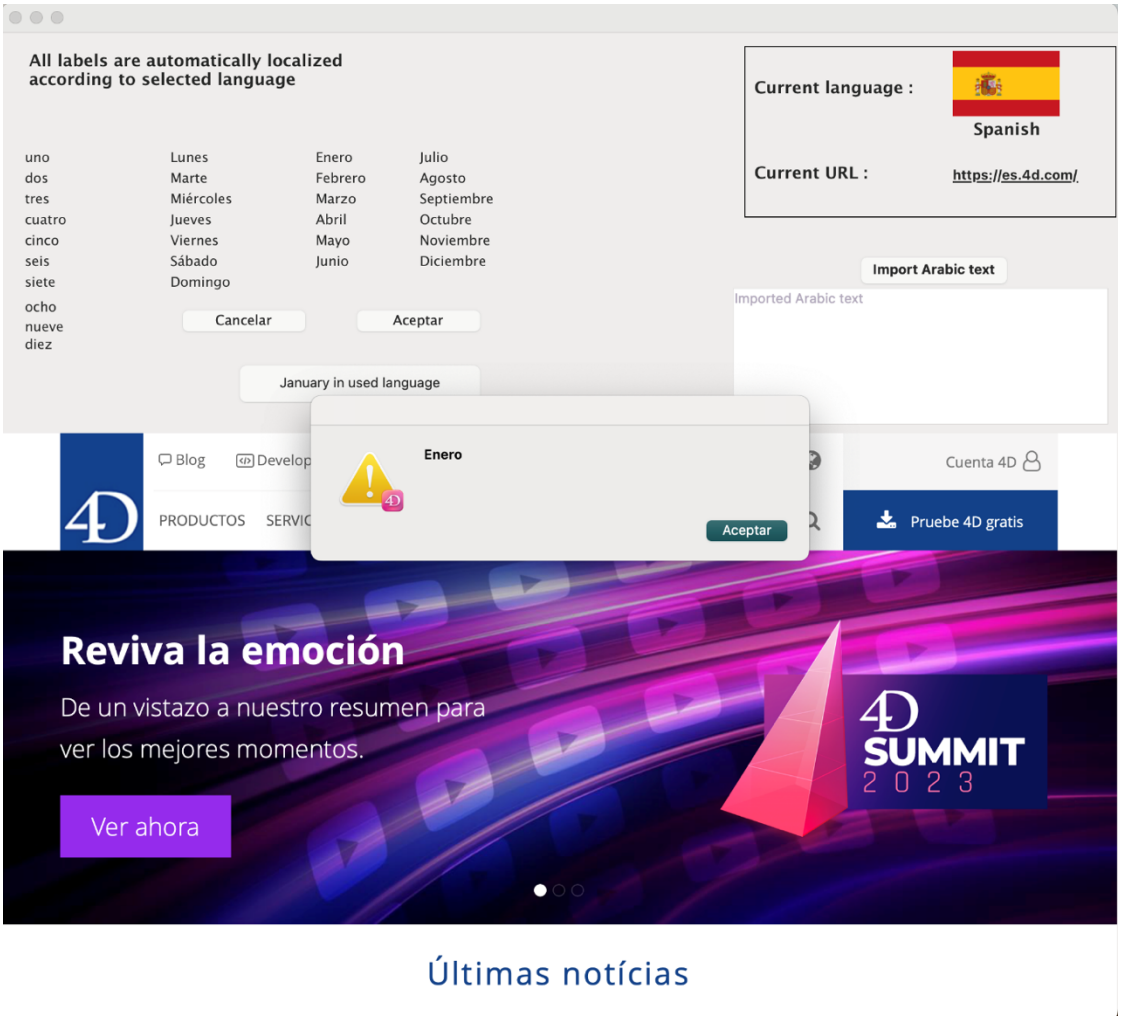
***Example*:** If resName is "month_1" and the current language is Spanish, the command will return "Enero."

```
<group resname="MonthNames">
   <trans-unit id="month_1" resname="month_1">
      <source>January</source>
      <target>Enero </target>
   </trans-unit>
```

After executing the following statement:

```
$valeurES:=Get localized string("month_1")
```

If the current language is Spanish, $valeurES will contain “Enero”.



Set Database Localization command for Current Language Control

SET DATABASE LOCALIZATION command sets the current language for the 4D application session, overriding the automatic language selection based on the .lproj folders for localized elements (such as text and images). While 4D typically selects the language automatically, this command allows for customization.

Note: Forms that are already loaded will not change language. Only new elements displayed after the command is executed will use the updated language settings.

The language is specified according to RFC 3066, ISO639 and ISO3166 norms using codes such as “fr” for French or “en-us” for American English.

For example, `SET DATABASE LOCALIZATION ("fr")` will set the interface to French.

By default, the command applies to all open databases and components, unless the optional * parameter is used to restrict the application to the calling database only.

The OK system variable indicates whether the command has been executed correctly (value 1 for success, 0 for failure).

The **Get database localization** command allows the developer to identify the current language.

Here is a simple example of how to use the **SET DATABASE LOCALIZATION** command to change the interface language based on the user's selection:

Scenario: The application prompts the user to select a language at startup. The interface will then display in the selected language.

```
// Ask user to select a language
CHOIX_LG:=Request("Please choose a language: fr for French, en for English, de for German :")

// Apply selected language
Case of
    : (CHOIX_LG="fr")
        SET DATABASE LOCALIZATION("fr")  // Change interface language to French
    : (CHOIX_LG="en")
        SET DATABASE LOCALIZATION("en")  // Change interface language to English
    : (CHOIX_LG="de")
        SET DATABASE LOCALIZATION("de")  // Change interface language to German
    Else
        SET DATABASE LOCALIZATION("en")  // By default, the interface is in English
End case
```

Details:

- Request("..."): Prompts the user to enter a language code (e.g., "fr" for French, "en" for English, "de" for German).
- Case ... End Case: A conditional structure used to handle different language choices based on user input.
- SET DATABASE LOCALIZATION("fr"): Sets the application language to French if the user enters "fr".
- SET DATABASE LOCALIZATION("en"): Sets the application language to English if the user enters "en".
- SET DATABASE LOCALIZATION("de"): Sets the application language to German if the user enters "de".

If the user enters an unrecognized code, the default language of the interface is set to English.

This example demonstrates how to change the application's language based on user input, ensuring a personalized and appropriate interface.

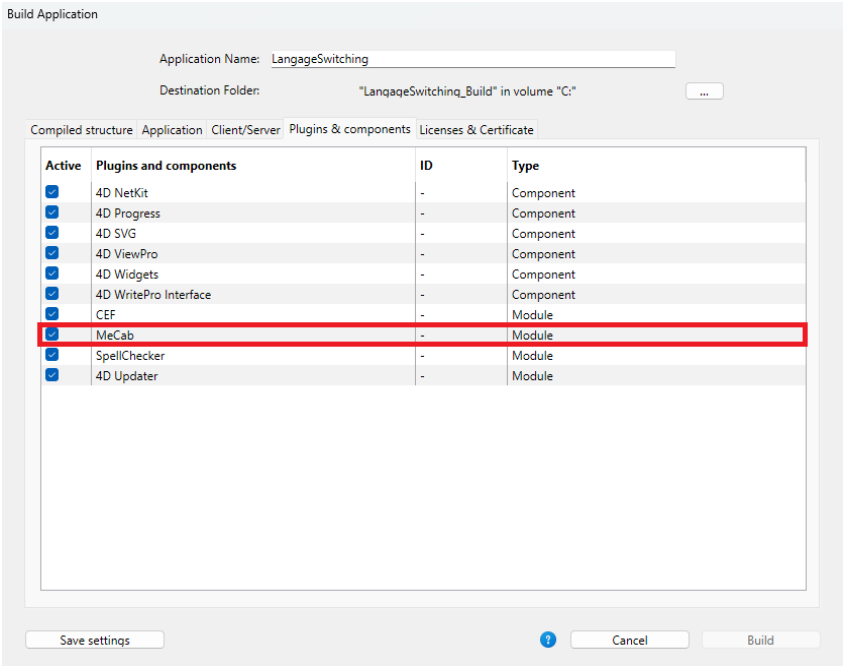
Asian and Arabic Languages in 4D Databases

1. Asian Languages

On Windows, the code editor includes an Input Method Editor (IME) to facilitate code entry on Japanese or Chinese systems.

Prior to version 19R5, a special version of 4D was provided to Japanese clients on macOS, as these customers were the only ones using MeCab. Now that MeCab can be easily removed from applications, it will be included for all users, and the special Japanese version of 4D will no longer be offered.

The MeCab library is included by default in all 4D applications on macOS. In previous versions, this library, specifically designed to handle Japanese text, was only available in the Japanese version of 4D on macOS. If this library is not needed in final applications, it can now be deselected.

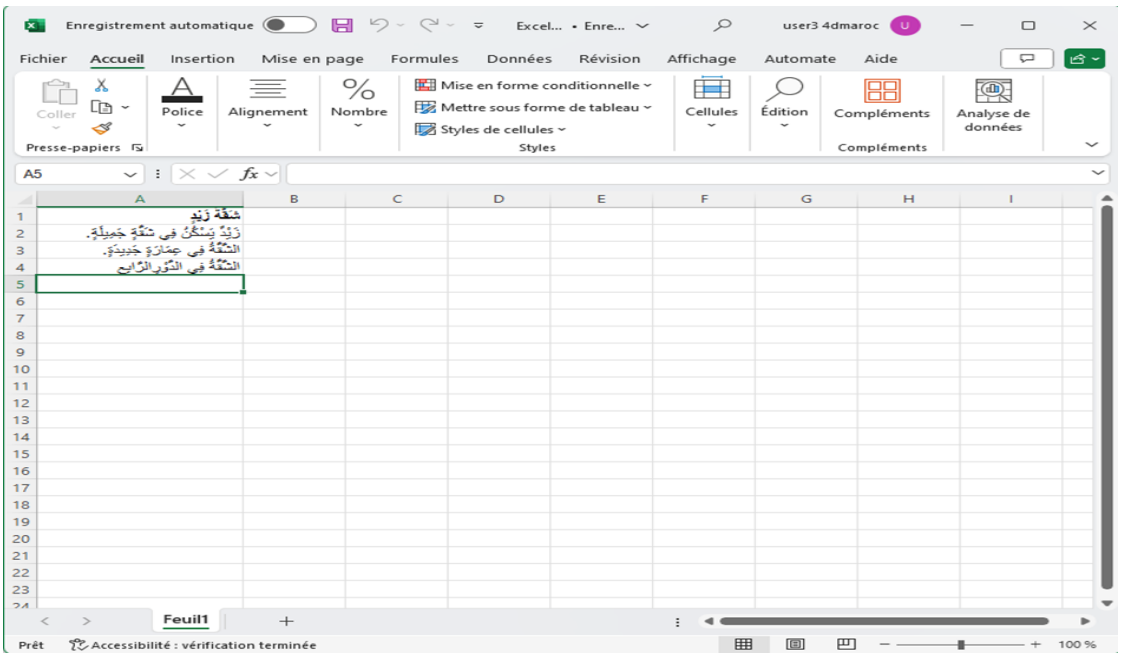


2. Arabic Language

4D supports the Arabic language, facilitating the input, processing, and display of Arabic text. This functionality enables the creation of applications tailored to the needs of Arabic-speaking users.

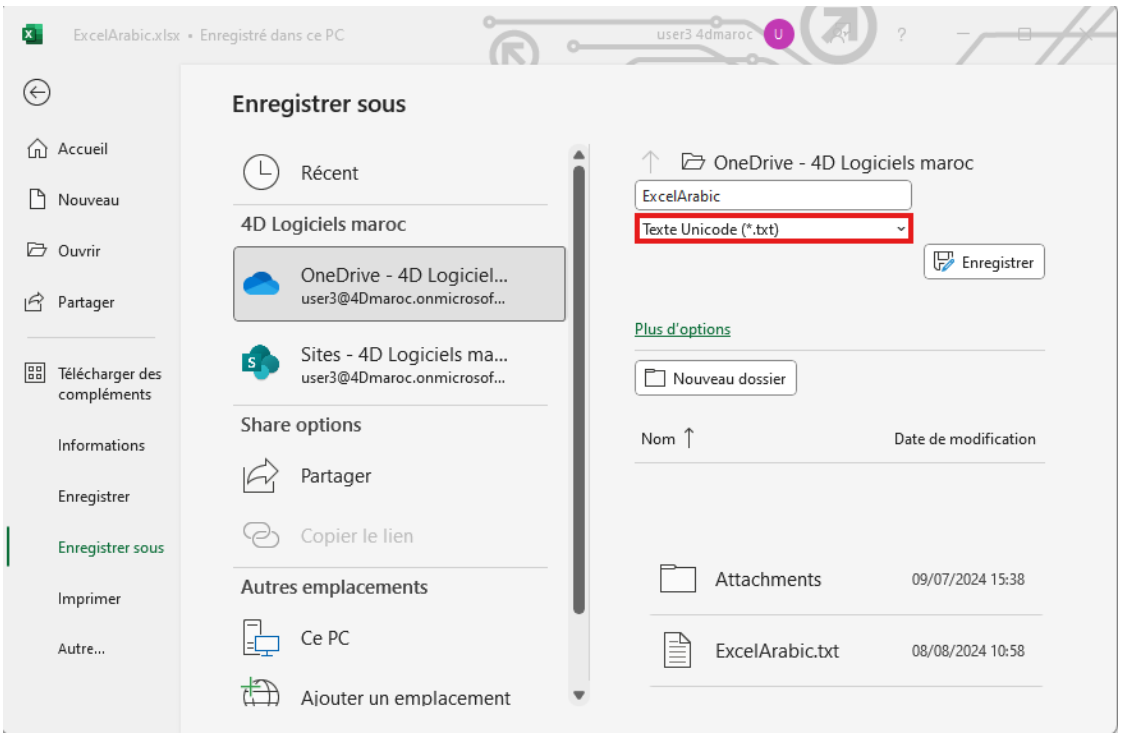
To import an Excel document written in Arabic into 4D and ensure that the characters are correctly displayed using Unicode encoding, follow these steps:

1. Prepare the Excel File in Unicode:
    - Open the Excel file containing Arabic data.



- Ensure that all data is correctly displayed in Arabic.
- Go to File > Save As.
- Prepare the Excel file in Unicode by saving it in the Unicode Text (*.txt) format.

Note: It is important to ensure that the language of the current data file in 4D is set to Arabic. This can be done by accessing the database properties under the Database tab, in the Text Comparison section.



- Provide a name (e.g. ExcelArabic) and save the file.

3. Importing Arabic Data into a 4D Database

To ensure proper handling and display of Arabic text in a 4D application, follow these steps to import and convert the data.

- Open an existing 4D database into which the data will be imported.
- Import the ExcelArabic.txt file into 4D, converting it to a blob.
- Save this blob in a field of a table.
- Convert the blob to text encoded in UTF-8 using the following code:

```
C_TEXT($path; $b2)
C_BLOB($b1)
$path:=Get 4D folder(Database folder)+"ExcelArabic1.txt"
DOCUMENT TO BLOB($path; $b1)
CREATE RECORD([Table_1])
[Table_1]Champ_2:=$b1
SAVE RECORD([Table_1])
$b2:=Convert to text([Table_1]Champ_2; "UTF-8")
Var1:=$b2
```

- Store this text in a variable and display it.



Conclusion

Creating multilingual applications is essential for reaching a global audience. Effective localization, including the use of the SET DATABASE LOCALIZATION command and XLIFF files, enhances user experience by adapting software to different languages and cultures. Modern resource management now favors XLIFF files and Unicode, ensuring broad character support and interoperability. Following the guidelines in this document will help developers ensure that 4D business applications provide a smooth and culturally relevant experience.